```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
```

Step 2: Load and Preprocess the Data

```
# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data() # Reshape data to include channel dimension and normalize pixel values X_train = X_train.reshape(-1, 28, 28, 1) / 255.0
X_test = X_test.reshape(-1, 28, 28, 1) / 255.0

# Convert labels to one-hot encoding
y_train, y_test = to_categorical(y_train), to_categorical(y_test)
```

Step 3: Define the Model

```
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)), # First convolutional layer MaxPooling2D((2, 2)), # First pooling layer Flatten(), # Flatten feature maps Dense(64, activation='relu'), # Fully connected layer Dense(10, activation='softmax') # Output layer for 10 classes ])
```

Step 4: Compile the Model

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

- **Optimizer**: Adam is a versatile optimizer combining the benefits of AdaGrad and RMSProp.
- **Loss Function**: Categorical Crossentropy is used for multi-class classification.
- **Metrics**: Accuracy is the performance metric.

Step 5: Train the Model

```
history = model.fit(X_train, y_train, epochs=10, batch_size=64, validation_split=0.2)
```

- **Epochs**: Specifies the number of complete passes through the dataset.
- **Batch Size**: Defines the number of samples processed before updating the model.
- **Validation Split**: Reserves 20% of the training data for validation.

Step 6: Evaluate the Model

```
loss, accuracy = model.evaluate(X_test, y_test)
print("Test Loss:", loss)
print("Test Accuracy:", accuracy)
```